

レポート課題 5

05-191009 奥村真司

問 1

動作例

```
# let x = 1+3;;
int
val x = 4
# let a = fun f -> fun g -> fun x -> f (g x);;
((('a4 -> 'a5) -> (('a3 -> 'a4) -> ('a3 -> 'a5)))
val a = <fun>
# let rec f x = if x = 0 then 0 else f (x-1) + x in f 10;;
int
- = 55
# let x = if true = 5 then true else false;;
Fatal error: exception ConstraintSolver.TyError
-----
# let x = if 3 = 5-2 then 1 else false;;
Fatal error: exception ConstraintSolver.TyError
-----
# let x = if 3 = 5-2 then 9 else 11;;
int
val x = 9
# let rec f x = if x = 0 then 0 else f (x-1) + x;;
(int -> int)
val f = <rec fun>
# f 10;;
int
- = 55
```

考察

講義スライドの通りの推論規則で今注目している式全体の型と制約を再帰的にもとめているのが `infer_expr` である。これでもとまった型と制約と、配布された `ConstraintSolver` の `unify`, `ty_subst` を用いて式の型を決定する。決定できなかった場合は `ConstraintSolver` の例外によって停止する。型がつく場合はついた型を `TySyntax.print_type` で表示するようにした。はじめ、`main` のループで、環境と同様に型環境も更新しつつ引き継いでやる必要があることを忘れて毎回空の型環境で型推論しており、`let x = 4;;x;;` とすると型エラーとなってしまうようになっていたので (`let` 式で宣言するチェックしかしていなかったのが気づけなかった。) `main` と `eval` を訂正し、型推論時は現在の型環境のもとで推論するように変えた。

発展 1

情報論理の授業で、 $\mathbb{N} \rightarrow \mathbb{N}$ の関数 f に f のコードを渡すような、自分に自分をいれるようなことをしたことがあったので、 $f\ f$ のようなことがしたくなることがあるのではと考えてみた。 f の型が $\alpha \rightarrow \beta$ で、 f を引数にとるとすると $\alpha = \alpha \rightarrow \beta$ という制約がつくが、これは代入すると無限に繰り替えされて、解けないので型がつかない。このような形を見たことある気がしたので思い出してみると、第 1 回の課題で不動点コンビネータについて調べていた時のラムダ計算だった。例えば Y コンビネータは

```
let y_comb = fun y -> ((fun x -> y (x x)) (fun x -> y (x x)));;
```

と書きたくなるが

```
# let y_comb = fun y -> ((fun x -> y (x x)) (fun x -> y (x x)));;
Error: This expression has type 'a -> 'b
      but an expression was expected of type 'a
      The type variable 'a occurs inside 'a -> 'b
```

のように型がつかない。これに型がつかないのは「型なしラムダ計算」だからのようである。いろいろと調べては見たが、詳しいことはわからなかった。(すみません) 理論上のことをそのまま落とし込んで実行したくても型がつかなくて実行できないのはつらいことのように感じる。実際第 1 回課題のチャーチ数の引き算も手でラムダ式で書いたことをそのまま書いてもなかなか型がつかなくて何時間も困った。(第 2 回で高ランク多相のヒントが与えられなければおそらくできなかったです)